



## TOPIC 05.0: BINARY SEARCH FOUNDATION & EVOLUTION

### REUSABLE BINARY SEARCH UTILITIES

#### Safe Mid & Boundary Helpers:

```
// Safe Midpoint calculation (prevents Integer overflow)
int mid = lo + (hi - lo) / 2;

// Lower Bound: First index where arr[i] ≥ target
int lowerBound(int[] arr, int target) {
    int lo = 0, hi = arr.length;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (arr[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

#### Feasibility Check Template (Search on Answer):

```
boolean feasible(int mid, int[] nums, int k) {
    int count = 1, sum = 0;
    for (int num : nums) {
        if (sum + num > mid) {
            count++; sum = num;
        } else sum += num;
    }
    return count ≤ k;
}
```

PAGE 1  
OF 10

#### ⚡ AHA MOMENT #1: SEARCH SPACE REDUCTION & BOUNDARY TRANSITION

Binary Search is NOT just an array search! It is a general **search-space reduction paradigm**. We evaluate the middle element 'mid' against a monotonic property (e.g. `F F F F T T T T`) to discard half ( $N/2$ ) of the search space in  $O(1)$  time, shrinking  $N$  elements to 1 in exactly  $\log_2(N)$  steps!

#### 🔄 PATTERN EVOLUTION: HOW ALGORITHMS EVOLVE

Linear Scan  $O(N)$  → Binary Search  $O(\log N)$  → Binary Search on Answer  $O(N \log M)$

- **Linear Scan:** Inspects elements sequentially from 0 to  $N-1$  ( $O(N)$  time).
- **Binary Search:** Halves search space interval at each step on monotonic inputs ( $O(\log N)$  time).
- **Binary Search on Answer:** Binary searches integer solution space  $[L, R]$  using feasibility checks in  $O(N \log M)$ !

#### 🔗 MONOTONIC TRANSITION BOUNDARY (F → T)

F

F

F

F

T ★

T

T

↑ Binary search pinpoints the FIRST TRUE transition boundary index in  $O(\log N)$ !

#### 1 OVERFLOW-SAFE MID FORMULA

Avoid integer overflow from `(left + right) / 2` by using:

```
int mid = left + (right - left) / 2;
```

#### 2 CLOSED INTERVAL [LEFT, RIGHT] RULE

With `left = 0, right = N - 1`, search space is inclusive `[left, right]`.  
Loop condition MUST be `while (left ≤ right)`.

🚀 **KEY TAKEAWAY:** For  $N = 1,000,000$ , Linear Scan takes 1,000,000 steps whereas Binary Search takes only ~20 steps!

1

CLASSIC BINARY SEARCH TEMPLATE

```
public int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left ≤ right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

2

FIRST OCCURRENCE (LOWER BOUND)

```
public int firstOccurrence(int[] nums, int target) {
    int left = 0, right = nums.length - 1, ans = -1;
    while (left ≤ right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) {
            ans = mid; right = mid - 1; // Record & search left!
        } else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return ans;
}
```

🔗

VISUAL TRACE: FIRST OCCURRENCE OF TARGET = 4

Input: `nums = [1, 2, 4, 4, 4, 5, 6]`, `Target = 4`

Step  
1

|       |   |   |           |   |   |       |
|-------|---|---|-----------|---|---|-------|
| 1     | 2 | 4 | 4 (mid=3) | 4 | 5 | 6     |
| 0 (L) | 1 | 2 | 3         | 4 | 5 | 6 (R) |

Match! Record `ans=3`, move `R = mid - 1 = 2`

Step  
2

|       |           |           |   |   |   |   |
|-------|-----------|-----------|---|---|---|---|
| 1     | 2 (mid=1) | 4 (idx 2) | 4 | 4 | 5 | 6 |
| 0 (L) | 1         | 2 (R)     | 3 | 4 | 5 | 6 |

2 < 4 → Move `L = mid + 1 = 2`.  
Final `ans = 2`!

💡 **AHA MOMENT:** Boundary search finds duplicate range boundaries in  $O(\log N)$  without linear scanning across duplicates!

- 💡 AHA MOMENT #4: SEARCHING SOLUTION RANGE  $[L, R]$  & 2D MATRIX FLATTENING
- **BS on Answer:** Binary search integer solution range  $[L, R]$  using boolean `isValid(mid)` helper.
  - **1D Matrix Mapping:** Convert 1D 'mid' to 2D coordinates in  $O(1)$ : `row = mid / cols` and `col = mid % cols` !

### 1 BS ON ANSWER JAVA TEMPLATE

```
public int bsOnAnswerTemplate(int[] nums, int limit) {
    int left = 1, right = getMax(nums), ans = right;
    while (left ≤ right) {
        int mid = left + (right - left) / 2;
        if (isValid(nums, mid, limit)) {
            ans = mid; right = mid - 1; // Try smaller!
        } else left = mid + 1;
    }
    return ans;
}
```

### 2 1D MATRIX FLATTENING (LC 74)

```
public boolean searchMatrix(int[][] matrix, int target) {
    int rows = matrix.length, cols = matrix[0].length;
    int left = 0, right = rows * cols - 1;
    while (left ≤ right) {
        int mid = left + (right - left) / 2;
        int val = matrix[mid / cols][mid % cols];
        if (val == target) return true;
        else if (val < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

### MATRIX SEARCH COMPLEXITY COMPARISON

| Matrix Structure                         | Algorithm Technique         | Time Complexity  | Space Complexity |
|------------------------------------------|-----------------------------|------------------|------------------|
| Fully Sorted (Row i end < Row i+1 start) | 1D Flattening Binary Search | $O(\log(R * C))$ | $O(1)$           |
| Rows & Cols Independently Sorted         | Top-Right Staircase Walk    | $O(R + C)$       | $O(1)$           |

🚀 **KEY TAKEAWAY:** BS on Answer converts hard optimization problems into feasibility checks in  $O(N \log M)$  time!

 **BINARY SEARCH INTERVIEW DECISION TREE**

**1 BUILT-IN JAVA APIS**

- `Arrays.binarySearch(arr, key)` returns index  $\geq 0$  if found, or `-(insertion_point) - 1` if missing!
- `Collections.binarySearch(list, key, comparator)` supports custom Object comparators.

**2 TOP 5 BINARY SEARCH BUGS**

1. Overflow in `(left + right)/2`.
2. Infinite stuck loop via `left = mid`.
3. Search on unsorted inputs.
4. Loop bound mismatches.
5. Missing empty input checks.

 **FAANG COACH TIP:** `Arrays.binarySearch()` returning `-insertionPoint - 1` lets you find insert index in 1 line!



## 1. LEETCODE 704 — BINARY SEARCH

Easy ★★★★★ Classic BS Amazon Meta

**Problem:**

Given a sorted array

nums

and

target

, return index or -1.

**Example:**

nums = [-1,0,3,5,9,12], target = 9

→ Output:

4

```
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

## 2. LEETCODE 35 — SEARCH INSERT POSITION

Easy ★★★★★ Lower Bound Google Apple

**Problem:**

Return index if target found. If not, return index where it would be inserted in order.

**Example:**

nums = [1,3,5,6], target = 5

→

2

.

target = 2

→

1

```
public int searchInsert(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return left; // left holds insertion index!
}
```

## 3. LEETCODE 374 — GUESS NUMBER HIGHER OR LOWER

Easy ★★★★★ API Evaluation Google

**Problem:**

Guess number between 1 and n using pre-defined

guess(int num)

API.

```
public int guessNumber(int n) {
    int left = 1, right = n;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int res = guess(mid);
        if (res == 0) return mid;
        else if (res < 0) right = mid - 1;
        else left = mid + 1;
    }
    return -1;
}
```

💡 **AHA MOMENT (LC 35):** On loop finish, 'left' pointer ALWAYS points to the exact insertion index!

#### 4. LEETCODE 74 — SEARCH A 2D MATRIX

Medium ★★★★★ 1D Flattening Meta Amazon

##### Problem:

Search target in  $M \times N$  integer matrix sorted left-to-right & top-to-bottom.

##### Example:

matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3

→

true

.

```
public boolean searchMatrix(int[][] matrix, int target) {
    int rows = matrix.length, cols = matrix[0].length;
    int left = 0, right = rows * cols - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int val = matrix[mid / cols][mid % cols];
        if (val == target) return true;
        else if (val < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

#### 5. LEETCODE 33 — SEARCH IN ROTATED SORTED ARRAY

Medium ★★★★★ Rotated Search Meta Amazon Google

##### Problem:

Search target in rotated sorted array of unique values.

##### Example:

nums = [4,5,6,7,0,1,2], target = 0

→

4

.

```
public int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) { // Left half sorted
            if (nums[left] <= target && target < nums[mid]) right = mid
            - 1;
            else left = mid + 1;
        } else { // Right half sorted
            if (nums[mid] < target && target <= nums[right]) left = mid
            + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

💡 **AHA MOMENT (LC 33):** At least ONE half (left or right) of a rotated sorted array is ALWAYS strictly sorted!

## 6. LEETCODE 153 — FIND MINIMUM IN ROTATED SORTED ARRAY


**Medium**
★★★★★
 Pivot Search
 [Meta](#)
[Amazon](#)

### Problem:

Given a rotated sorted array

nums

of unique elements, return minimum element.

### Example:

nums = [3,4,5,1,2]

→

1

.

```
public int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[right]) left = mid + 1;
        else right = mid;
    }
    return nums[left];
}
```

## 7. LEETCODE 875 — KOKO EATING BANANAS


**Medium**
★★★★★
 BS on Answer
 [Google](#)
[Meta](#)

### Problem:

Min eating speed

k

to eat all banana piles within

h

hours.

### Example:

piles = [3,6,7,11], h = 8

→

4

.

```
public int minEatingSpeed(int[] piles, int h) {
    int left = 1, right = 0;
    for (int p : piles) right = Math.max(right, p);
    int res = right;
    while (left ≤ right) {
        int k = left + (right - left) / 2;
        long hours = 0;
        for (int p : piles) hours += (p + k - 1) / k;
        if (hours ≤ h) {
            res = k; right = k - 1; // Try slower
        } else left = k + 1;
    }
    return res;
}
```


**AHA MOMENT (LC 875):** Integer ceiling division formula:  $(p + k - 1) / k$  avoids floating point calculations!

🔥 Medium ★★★★★ Slope Search [Meta](#) [Amazon](#)**Problem:**

Find index of any peak element strictly greater than its neighbors.

**Example:**

```
nums = [1,2,3,1]
```

```
→
```

```
2
```

```
.
```

```
public int findPeakElement(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] < nums[mid + 1]) left = mid + 1;
        else right = mid;
    }
    return left;
}
```

## 9. LEETCODE 981 — TIME BASED KEY-VALUE STORE

🔥 Medium ★★★★★ List BS Search [Google](#) [Meta](#)**Problem:**

Store key-value pairs with timestamps and retrieve value with

```
timestamp_prev ≤ timestamp
```

```
.
```

```
class TimeMap {
    private Map<String, List<Pair>> map = new HashMap<>();
    public void set(String key, String value, int timestamp) {
        map.computeIfAbsent(key, k → new ArrayList<>()).add(new
        Pair(timestamp, value));
    }
    public String get(String key, int timestamp) {
        if (!map.containsKey(key)) return "";
        List<Pair> list = map.get(key);
        int left = 0, right = list.size() - 1; String res = "";
        while (left ≤ right) {
            int mid = left + (right - left) / 2;
            if (list.get(mid).time ≤ timestamp) {
                res = list.get(mid).val; left = mid + 1;
            } else right = mid - 1;
        }
        return res;
    }
    private static class Pair { int time; String val; Pair(int t, String
    v){time=t;val=v;}}
}
```

🚩 **KEY TAKEAWAY:** Peak element search moves towards ascending slope (`nums[mid] < nums[mid+1]`) because a peak **MUST** exist in that direction!



## 10. LEETCODE 410 — SPLIT ARRAY LARGEST SUM

Hard ★★★★★ BS on Answer Google Amazon

### Problem:

Split array into

k

non-empty contiguous subarrays minimizing largest sum.

```
public int splitArray(int[] nums, int k) {
    int left = 0, right = 0;
    for (int n : nums) { left = Math.max(left, n); right += n; }
    int res = right;
    while (left ≤ right) {
        int mid = left + (right - left) / 2;
        if (canSplit(nums, k, mid)) { res = mid; right = mid - 1; }
        else left = mid + 1;
    }
    return res;
}

private boolean canSplit(int[] nums, int k, int maxSum) {
    int count = 1, currentSum = 0;
    for (int n : nums) {
        if (currentSum + n > maxSum) { count++; currentSum = n; }
        else currentSum += n;
    }
    return count ≤ k;
}
```

## 11. LEETCODE 4 — MEDIAN OF TWO SORTED ARRAYS

Hard ★★★★★ Partition BS Google Meta

### Problem:

Return median of two sorted arrays in  $O(\log(M+N))$  time!

```
public double findMedianSortedArrays(int[] nums1, int[] nums2) {
    if (nums1.length > nums2.length) return
    findMedianSortedArrays(nums2, nums1);
    int m = nums1.length, n = nums2.length, half = (m + n + 1) / 2;
    int left = 0, right = m;
    while (left ≤ right) {
        int i = left + (right - left) / 2, j = half - i;
        int maxL1 = (i == 0) ? Integer.MIN_VALUE : nums1[i - 1];
        int minR1 = (i == m) ? Integer.MAX_VALUE : nums1[i];
        int maxL2 = (j == 0) ? Integer.MIN_VALUE : nums2[j - 1];
        int minR2 = (j == n) ? Integer.MAX_VALUE : nums2[j];
        if (maxL1 ≤ minR2 && maxL2 ≤ minR1) {
            if ((m + n) % 2 == 1) return Math.max(maxL1, maxL2);
            return (Math.max(maxL1, maxL2) + Math.min(minR1, minR2)) /
            2.0;
        } else if (maxL1 > minR2) right = i - 1;
        else left = i + 1;
    }
    return 0.0;
}
```

 **FAANG COACH TIP:** For Median of Two Sorted Arrays, binary search on the SMALLER array to guarantee  $O(\log(\min(M,N)))$  complexity!

### ★ THE 4 GOLDEN LAWS OF BINARY SEARCH

- Overflow-Safe Mid:** Always calculate `int mid = left + (right - left) / 2;`.
- Inclusive Bounds Rule:** With `left = 0, right = N - 1`, loop MUST be `while (left ≤ right)` and update `left = mid + 1`, `right = mid - 1`.
- Boundary Record Trick:** For first/last occurrence, record `ans = mid` when matched and continue searching leftward/rightward.
- Binary Search on Answer:** Search integer solution range  $[L, R]$  with boolean `isValid(mid)` helper!

### 🚫 WHEN BINARY SEARCH FAILS (DO NOT USE!)

- **Unsorted Input Array:** Input is unsorted and index order must be preserved → Use **HashMap!**
- **Non-Monotonic Predicate:** `isValid(x)` fluctuates unpredictably → Use **Dynamic Programming!**

### 📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

|                                                                                                                     |                                                                                                                      |                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
|  <b>Easy</b> LC 704 Binary Search   |  <b>Easy</b> LC 35 Search Insert    |  <b>Easy</b> LC 374 Guess Number    |
|  <b>Med</b> LC 74 Search 2D Matrix  |  <b>Med</b> LC 33 Search Rotated    |  <b>Med</b> LC 153 Find Min Rotated |
|  <b>Med</b> LC 875 Koko Bananas     |  <b>Med</b> LC 162 Find Peak        |  <b>Med</b> LC 981 Time Key Value   |
|  <b>Hard</b> LC 410 Split Array Max |  <b>Hard</b> LC 4 Median Two Sorted |                                                                                                                        |

👉 Next Master Topic: [Topic 06 → Stack & Queues Masterclass](#)

🚩 **TOPIC 05 BINARY SEARCH COMPLETE!** Turn the page to **Topic 06: Stack & Queues Masterclass!**